

October 2025

1. Primera entrega

1.1. Contextualización

Solactive es un proveedor de índices con sede en Alemania que opera a nivel mundial y crece a un ritmo acelerado. Desde 2007, desarrolla soluciones de índices a medida y multiclase de activos para ETF y otros productos de inversiones indexados, con los principales bancos de inversión y gestores de activos mundiales como clientes.

La serie de índices de referencia global solactive ofrece un marco integral de índices que abarcan el mercado bursátil global. Estos índices pueden utilizarse directamente en un producto, como referencia o como universo de partida para un índice desarrollado.

1.2. Formulación del problema

1.2.1. Problema central

El segmento de Price Data de Solactive enfrenta limitaciones en la validación de precios de cierre de activos financieros. El proceso actual exige coincidencia exacta entre los tres proveedores de datos (ICE como primario; FactSet y EDI como secundarios) para clasificar un precio como VALIDATED; si solo coincide en dos, se marca SEMI_VALIDATED y, si no coincide, UNVALIDATED. Esta rigidez produce “falsos no válidos”: diferencias menores en unidades (p. ej., dólares vs. centavos), formatos o decimales insignificantes provocan que precios objetivamente correctos se clasifiquen como UNVALIDATED, interrumpiendo el flujo hacia el cálculo de índices.

1.2.2. Relevancia técnica

La ausencia de reglas de validación flexibles (normalización de unidades, tolerancias numéricas, redondeos consistentes y armonización de formatos) obliga a recalcular índices de forma manual para corregir discrepancias triviales. Esto aumenta tiempos de procesamiento, eleva la carga operativa y reduce la confiabilidad del algoritmo de validación. Un sistema con reglas parametrizables y pre-procesamiento estandarizado mejoraría la calidad de datos de entrada, permitiría trazabilidad/auditabilidad de las decisiones de validación y reduciría retrabajos, con impacto directo en el rendimiento del índice.

1.2.3. Relevancia social/empresarial

Los retrasos y retrabajos repercuten en los plazos de entrega comprometidos con clientes institucionales, afectando la satisfacción, la confianza y la percepción de calidad del servicio. Minimizar los falsos UNVALIDATED y automatizar correcciones reduce

costos operativos, estabiliza los tiempos de publicación de índices y disminuye incidentes con contrapartes que emiten productos financieros basados en dichos índices. En suma, abordar el problema preserva la reputación de Solactive, mejora la experiencia de vendedores y fortalece la competitividad en un mercado donde la precisión y la puntualidad son críticas. Si el precio coincide en los tres vendedores, se considera VALIDATED.

1.3. Objetivos del Proyecto

Desarrollar una solución tecnológica soportada en una base de datos relacional y no relacional que facilite la validación, carga y depuración de los datos de precios de activos provenientes de distintos vendedores.

- Establecer un conjunto de reglas para el tratamiento de los datos invalidados
- Reducir la necesidad de cálculos manuales
- Garantizar la consistencia de la información enviada a los clientes

1.4. Alcance

- Delimitación funcional:
 - El proyecto busca optimizar el tiempo del segmento Equity Core en el proceso de verificación de datos.
 - Se propone automatizar la comparación de la información del cierre de mercado proporcionada por los tres proveedores: ICE, FactSet y EDI.
 - La herramienta verificará que los datos sean idénticos en al menos dos de los tres proveedores.
 - En caso de detectar una inconsistencia, notificará automáticamente al segmento Equity Core.
- Delimitación técnica:
 - La limpieza y transformación de los datos se realizarán utilizando Visual Studio Code como entorno de desarrollo.
 - El proyecto se implementará en Python, empleando librerías como Pandas y NumPy para la manipulación y análisis de la información.
 - La base de datos relacional se empleará para almacenar los registros procesados y mantener la trazabilidad de los resultados.

- Esta configuración técnica permitirá un manejo automatizado, eficiente y confiable de los datos.
- Alcances excluidos:
 - No contempla la integración directa con plataformas de información financiera ni con sistemas en tiempo real.
 - No se desarrollará una interfaz gráfica avanzada, ya que el objetivo principal es la automatización interna de los procesos, no el diseño visual.
 - No se incluirán funciones adicionales distintas a la comparación de los datos de cierre.
- Alcances futuros:
 - Se proyecta la integración del sistema con servicios en la nube, con el fin de mejorar su escalabilidad, rendimiento y disponibilidad.

1.5. Metodología de desarrollo

Hemos creado un tablero Kanban en Trello que demuestra nuestro trabajo como repartición de tareas. El tablero está dividido en las siguientes columnas o listas:

- **Backlog:** Esta es nuestra lista de tareas pendientes. Aquí se encuentran todas las ideas y funcionalidades que queremos implementar en futuras entregas. De aquí "jalamos" las tareas a la columna "To do" cuando decidimos trabajarlas.
- **To do:** Esta columna contiene las tareas que están planificadas para ser realizadas en esta entrega.
- **Doing:** Aquí están las tareas en las que estamos trabajando activamente en este momento.
- **Done:** Esta columna es para las tareas que ya están terminadas. Una vez que una tarea se mueve a esta lista, sabemos que ya no hay que preocuparse por ella.

Link Trello: Tablero Kanban Equipo 4

1.6. Requisitos

1.6.1. Requisitos funcionales de alto nivel

Código	Requisitos funcionales
RQF001	Nombre: Extracción, Verificación y Consolidación de Datos (EVC)
	Descripción: Extracción de datos de los diferentes vendors de información, verificación de información y consolidación en base de datos para validación
	Usuarios: División de Price Data

Código	Requisitos funcionales
RQF002	Nombre: Gestión y Validación de Datos (GDV)
	Descripción: Uso de base de datos resultado de EVC donde se utilicen las reglas de validación para brindar el estado de validación de precios de cierre
	Usuarios: División de Price Data

Código	Requisitos funcionales
RQF003	Nombre: Visualización de Datos (DV)
	Descripción: Integración de resultados a una aplicación de visualización para facilitar la interpretación de datos y su identificación sencilla, entrega de reportes y trazabilidad
	Usuarios: División de Price Data

1.6.2. Requisitos funcionales de bajo nivel

Código	Requisitos funcionales de bajo nivel
RQF_LOW01	Nombre: Archivos soportados
	Descripción: El sistema de ECV debe soportar archivos de tipo CSV, XLSX y TXT
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW02	Nombre: Verificación de disponibilidad de datos de vendors
	Descripción: El sistema ECV debe comprobar que los datos de precio cargados de cada vendor sean o numéricos o nulos para la fecha establecida antes de iniciar el proceso de validación.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW03	Nombre: Verificación de información disponible
	Descripción: El sistema ECV debe reconocer los datos disponibles en la información brindada por cada vendor y verificar su consistencia con la información de los demás proveedores
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW04	Nombre: Depuración de datos
	Descripción: El sistema ECV debe realizar la depuración de las columnas que no sean útiles para la validación
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW05	Nombre: Estandarización de información de precios
	Descripción: El sistema ECV debe consultar el precio del mismo activo en las tablas asociadas a cada vendor y asegurarse de que tengan la misma denominación (e.j. dólar, centavos), de no tenerla, debe ajustar el precio entre tablas para que así sea
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW06	Nombre: Consolidación de información
	Descripción: El sistema ECV debe extraer toda la información previamente depurada consolidarla en una única tabla
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW07	Nombre: Exporte inicial de información
	Descripción: El sistema ECV debe permitir al usuario realizar un exporte de la base de datos depurada y estandarizada antes de realizar la validación de datos si el mismo así lo desea a un archivo nombrado de forma estándar en formato CSV
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW08	Nombre: Eliminación de información inicial
	Descripción: El sistema ECV debe permitir al usuario realizar la eliminación total de la información previa al exporte inicial si este así lo desea
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW09	Nombre: Consulta de archivo estándar
	Descripción: El sistema GDV debe poder consultar la información de los archivos nombrados de forma estándar en formato CSV
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW10	Nombre: Creación, modificación y eliminación de reglas de validación
	Descripción: El sistema GDV debe permitir al usuario crear, modificar y eliminar las reglas de validación
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW11	Nombre: Jerarquización de vendors
	Descripción: El sistema GDV debe permitir al usuario consultar la jerarquía de importancia entre vendors de información
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW12	Nombre: Reglas de validación
	Descripción: El sistema GDV debe integrar las reglas de validación desarrolladas previo a su uso
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW13	Nombre: Comparación de datos
	Descripción: El sistema GDV debe permitir la comparación entre la información disponible
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW14	Nombre: Validación de precios
	Descripción: El sistema GDV debe realizar la comparación de datos de precios de cierre entre los 3 vendors y asignarle su categoría correspondiente
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW15	Nombre: Validación Manual
	Descripción: El sistema GDV debe permitir el mecanismo de validación manual en caso de ser necesario
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW16	Nombre: Creación de métricas de rendimiento de validación
	Descripción: El sistema GDV debe realizar el conteo de cada tipo de validación de datos
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW17	Nombre: Registro de resultados de validación
	Descripción: El sistema GDV debe guardar automáticamente el resultado de cada proceso de validación, incluyendo el estado asignado y la fecha en que se realizó.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW18	Nombre: Registro de errores detectados
	Descripción: El sistema GDV debe almacenar los errores identificados durante la validación, indicando el tipo de error, el activo afectado y la fecha del incidente.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW19	Nombre: Consulta de errores registrados
	Descripción: El sistema GDV debe permitir consultar los errores guardados, filtrando por tipo, fecha o proveedor, para facilitar su revisión y corrección.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW20	Nombre: Filtro de datos
	Descripción: El sistema GDV debe permitir al usuario aplicar filtros de búsqueda específicos para la información disponible
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW21	Nombre: Identificación de inconsistencias de información
	Descripción: El sistema GDV debe mostrar al usuario las inconsistencias en los precios invalidados y su diferencia frente a las reglas de validación
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW22	Nombre: Ajustes por eventos corporativos
	Descripción: El sistema GDV debe consultar/aplicar factores de ajuste por corporate actions (splits, dividendos en especie, consolidaciones) a los precios históricos del día de corte antes de validar, manteniendo el precio “crudo” y el “ajustado”.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW23	Nombre: Notificaciones y enrutamiento de incidencias
	Descripción: El sistema GDV debe permitir revertir una validación manual, restaurar el estado anterior y re-ejecutar reglas con un solo clic, dejando trazabilidad (quién, cuándo, motivo) y evitando efectos colaterales no auditados.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW24	Nombre: Reversión de validaciones manuales
	Descripción: El sistema GDV debe permitir revertir una validación manual y restaurar su estado anterior, registrando la acción con el usuario que la ejecutó, la fecha y el motivo de reversión.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW25	Nombre: Consulta de datos para visualización
	Descripción: El sistema DV debe poder consultar la base de datos final
	Usuarios: División de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW26	Nombre: Panel de validaciones
	Descripción: El sistema DV debe mostrar el consolidado validaciones por tipo
	Usuarios: División de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW27	Nombre: Filtros dinámicos
	Descripción: El sistema DV debe permitir filtrar por vendedor, tipo de activo y rango de timestamp
	Usuarios: División de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW28	Nombre: Comparación entre vendedores
	Descripción: El sistema DV debe mostrar las estadísticas de inconsistencias de cada vendedor y su frecuencia
	Usuarios: División de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW29	Nombre: Métricas históricas de desempeño de validación
	Descripción: El sistema DV debe mostrar el histórico de validaciones y su evolución a lo largo del tiempo
	Usuarios: División de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW30	Nombre: Edición controlada de validaciones
	Descripción: El sistema GDV debe permitir modificar los resultados de validaciones manuales de forma controlada, asegurando que los cambios queden registrados con trazabilidad y sin afectar otras validaciones.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW31	Nombre: Consulta de datos validados históricos
	Descripción: El sistema debe permitir consultar versiones anteriores de los datos validados, conservadas en la base de datos, para comparar cambios realizados en diferentes fechas.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW32	Nombre: Panel de auditoría visual
	Descripción: El sistema debe permitir visualizar la trazabilidad de un precio (origen, reglas aplicadas y validación final).
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW33	Nombre: Eliminación controlada de registros
	Descripción: El sistema debe permitir eliminar registros de validación únicamente tras confirmación del usuario, evitando eliminaciones accidentales o no autorizadas.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW34	Nombre: Alertas visuales
	Descripción: El sistema debe resaltar los activos con inconsistencias o validaciones incompletas y su timestamp.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW35	Nombre: Exportación desde Power BI
	Descripción: El sistema debe permitir la exportación de tablas y gráficos desde los dashboards en formatos PDF y Excel.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW36	Nombre: Alertas automáticas por fallos de recolección
	Descripción: El sistema debe enviar alertas automáticas al segmento de Equity Core cuando falle la recolección de datos de algún vendor
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW37	Nombre: Registro de acciones de usuario
	Descripción: El sistema debe guardar en una tabla las acciones principales realizadas por cada usuario (creación, validación, eliminación o modificación), incluyendo fecha y hora.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW38	Nombre: Respaldo previo a modificaciones
	Descripción: El sistema debe generar una copia de los registros antes de ejecutar procesos que modifiquen o eliminen datos, para garantizar su recuperación en caso de error.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW39	Nombre: Gestión de incidencias de validación
	Descripción: El sistema GDV debe permitir crear una incidencia cuando un precio quede UNVALIDATED, registrar comentarios/adjuntos, y moverla por estados (Abierta, En Curso, Resuelta, Cerrada) con trazabilidad.
	Usuarios: División de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW40	Nombre: Reversión y edición controlada de validaciones
	Descripción: El sistema GDV debe permitir revertir una validación manual, restaurar el estado anterior y re-ejecutar reglas con un solo clic.
	Usuarios: División de Price Data

1.6.3. Requisitos no funcionales

Código	Requisitos no funcionales
NoRQF01	Nombre: Acceso seguro a configuración
	Descripción: El sistema debe garantizar que solo usuarios autenticados y autorizados puedan acceder a la configuración de reglas de validación.

Código	Requisitos no funcionales
NoRQF02	Nombre: Comunicación cifrada con vendors
	Descripción: Toda la comunicación entre el sistema y las fuentes de datos externas (vendors) debe estar cifrada mediante protocolos seguros.

Código	Requisitos no funcionales
NoRQF03	Nombre: Trazabilidad y control de registros
	Descripción: Los registros de validación deben almacenarse con control de acceso y trazabilidad de cambios para auditoría.

Código	Requisitos no funcionales
NoRQF04	Nombre: Documentación clara del código
	Descripción: El código y las reglas de validación deben estar documentados con comentarios claros y actualizados.

Código	Requisitos no funcionales
NoRQF05	Nombre: Agilidad en incorporación de reglas
	Descripción: La incorporación de una nueva regla de validación no debe requerir más de 4 horas de trabajo técnico.

Código	Requisitos no funcionales
NoRQF06	Nombre: Actualizaciones con mínima interrupción
	Descripción: Las actualizaciones del sistema deben poder realizarse sin interrumpir más de 15 minutos de servicio.

Código	Requisitos no funcionales
NoRQF07	Nombre: Disponibilidad mínima
	Descripción: El sistema debe estar disponible para los usuarios al menos en un 99.5 % del tiempo mensual.

Código	Requisitos no funcionales
NoRQF08	Nombre: Continuidad ante fallo de un vendor
	Descripción: En caso de fallo de un vendor, el sistema debe continuar operando con los proveedores restantes sin interrumpir el flujo de validación.

Código	Requisitos no funcionales
NoRQF09	Nombre: Recuperación máxima
	Descripción: El tiempo máximo de recuperación tras una caída no debe superar las 2 horas

Código	Requisitos no funcionales
NoRQF10	Nombre: Interfaz accesibles configuración
	Descripción: La interfaz de usuario debe permitir configurar nuevas reglas de validación sin necesidad de conocimientos de programación avanzados.

Código	Requisitos no funcionales
NoRQF11	Nombre: Mensajes de error claros y comprensibles
	Descripción: Los mensajes de error o alerta deben ser claros y comprensibles, indicando el motivo de la invalidación y la posible causa (ej. diferencias en unidades, decimales, formato).

Código	Requisitos no funcionales
NoRQF12	Nombre: Procesamiento masivo
	Descripción: El sistema debe procesar y validar al menos 10,000 precios de activos en menos de 1 minuto en condiciones normales de operación.

Código	Requisitos no funcionales
NoRQF13	Nombre: Tiempo de respuesta por activo
	Descripción: El tiempo máximo de respuesta para la validación de un lote de precios no debe superar los 2 segundos por activo.

1.7. Historias de Usuario para RQF seleccionados.

Historia de Usuario	
Número: 01	Usuario: Analista de Price Data de Solactive
Nombre Historia: Carga de datos desde diferentes formatos	
Descripción: “Me gustaría que el sistema soportara distintos tipos de archivos como CSV, TXT o XLXS para poder cargar distintos formatos de los venders sin tener que convertirlos manualmente.”	

Historia de Usuario	
Número: 02	Usuario: Analista de Price Data de Solactive
Nombre Historia: Verificación de disponibilidad de datos de venders	
Descripción: “Quiero que el sistema compruebe si existen datos cargados de cada vendedor antes de iniciar la validación, para evitar errores cuando falte información y no perder tiempo revisando manualmente.”	

Historia de Usuario	
Número: 03	Usuario: Analista de Price Data de Solactive
Nombre Historia: Verificación de consistencia entre venders	
Descripción: “Si el sistema pudiera verificar la información disponible entre los distintos venders y detectar inconsistencias, se aseguraría la coherencia de los datos antes del proceso de validación.”	

Historia de Usuario	
Número: 04	Usuario: Analista de Price Data de Solactive
Nombre Historia: Depuración automática de columnas innecesaria	
Descripción: “Si el sistema eliminara de forma automática las columnas que no son útiles para la validación, se optimizaría tanto el tiempo como la base de datos y se reduciría la carga de procesamiento.”	

Historia de Usuario	
Número: 05	Usuario: Analista de Price Data de Solactive
Nombre Historia: Estandarización de información de precios	
Descripción: “Sería de mucha utilidad si el mismo sistema ajustara las diferencias de denominación entre tablas como dólares y centavos, los precios de un mismo activo estarían alineados y la validación sería más precisa, además de que nos ahorraría el tener que hacerlo de forma manual”	

Historia de Usuario	
Número: 06	Usuario: Analista de Price Data de Solactive
Nombre Historia: Consolidación de información depurada	
Descripción: “Como analista, quiero que toda la información depurada se consolide en una única tabla, para facilitar la validación y evitar duplicidades.”	

Historia de Usuario	
Número: 07	Usuario: Analista de Price Data de Solactive
Nombre Historia: Exportación inicial de información depurada	
Descripción: “Necesito poder exportar la base de datos depurada en formato CSV antes de validar los datos, para conservar un respaldo del estado inicial”	

Historia de Usuario	
Número: 08	Usuario: Analista de Price Data de Solactive
Nombre Historia: Eliminación total de información previa	
Descripción: “Quiero tener la opción de eliminar completamente la información cargada antes del exporte inicial, para mantener un entorno de trabajo limpio y sin duplicaciones.”	

Historia de Usuario	
Número: 09	Usuario: Analista de Price Data de Solactive
Nombre Historia: Consulta de archivo estándar	
Descripción: “Facilitaría mucho que el sistema pueda consultar archivos nombrados de forma estándar en formato CSV, para mantener la organización y agilizar el proceso de validación.”	

Historia de Usuario	
Número: 10	Usuario: Analista de Price Data de Solactive
Nombre Historia: Gestión de reglas de validación	
Descripción: “Me gustaría crear, modificar y eliminar reglas de validación directamente desde el sistema, para poder ajustar los criterios según las necesidades del proceso.”	

Historia de Usuario	
Número: 11	Usuario: Analista de Price Data de Solactive
Nombre Historia: Jerarquización de proveedores de información	
Descripción: “Como analista, quiero poder ver jerarquía de importancia entre los vendors, para saber de quien son los datos a los que se les está dando prioridad.”	

Historia de Usuario	
Número:12	Usuario: Analista de Price Data de Solactive
Nombre Historia: Integración de reglas de validación	
Descripción: “Necesito que el sistema integre las reglas de validación definidas previamente, para que se apliquen automáticamente en cada proceso.”	

Historia de Usuario	
Número: 13	Usuario: Analista de Price Data de Solactive
Nombre Historia: Comparación entre conjuntos de datos	
Descripción: “Como analista, quiero que el sistema permita comparar la información de los distintos vendors, para detectar discrepancias en los precios de los activos.”	

Historia de Usuario	
Número: 14	Usuario: Analista de Price Data de Solactive
Nombre Historia: Validación automática de precios de cierre	
Descripción: “Nos que el sistema compare los precios de cierre entre los tres vendors y asigne su categoría: VALIDATED, SEMIVALIDATED o UNVALIDATED, para automatizar el proceso de validación.”	

Historia de Usuario	
Número: 15	Usuario: Analista de Price Data de Solactive
Nombre Historia: Validación manual de precios	
Descripción: “Además de lo automático sería bueno poder realizar validaciones manuales cuando sea necesario, para corregir casos especiales que el sistema no pueda resolver automáticamente.”	

Historia de Usuario	
Número: 16	Usuario: Jefe de Price Data de Solactive
Nombre Historia: Métricas de desempeño del proceso de validación	
Descripción: “Como analista, quiero que el sistema realice el conteo de cada tipo de validación de datos, para poder analizar el rendimiento y detectar patrones en el proceso de validación.”	

Historia de Usuario	
Número: 17	Usuario: Analista de Price Data de Solactive
Nombre Historia: Registro automático de resultados de validación	
Descripción: “Me gustaría que el sistema guardara automáticamente el resultado de cada validación con su fecha y estado, para tener un registro confiable del proceso sin tener que hacerlo manualmente.”	

Historia de Usuario	
Número: 18	Usuario: Analista de Price Data de Solactive
Nombre Historia: Registro de errores detectados	
Descripción: “Necesito que el sistema guarde los errores que se detectan durante la validación, indicando el tipo de error y el activo afectado, para poder revisarlos y corregirlos después.”	

Historia de Usuario	
Número: 19	Usuario: Analista de Price Data de Solactive
Nombre Historia: Consulta de errores registrados	
Descripción: “Quiero poder consultar los errores que el sistema haya registrado, filtrando por tipo, fecha o proveedor, para identificar rápidamente los problemas más frecuentes.”	

Historia de Usuario	
Número: 20	Usuario: Analista de Price Data de Solactive
Nombre Historia: Filtro de datos	
Descripción: “Quiero aplicar filtros de búsqueda específicos en la información, para localizar rápidamente los registros que necesito revisar.”	

Historia de Usuario	
Número: 21	Usuario: Analista de Price Data de Solactive
Nombre Historia: Detección de inconsistencias en precios	
Descripción: “Sería bueno que el sistema me muestre las inconsistencias en los precios invalidados y su diferencia respecto a las reglas de validación, para analizarlas fácilmente.”	

Historia de Usuario	
Número: 22	Usuario: Analista de Price Data de Solactive
Nombre Historia: Ajuste de precios por eventos corporativos	
Descripción: “Como analista, quiero que el sistema aplique factores de ajuste por eventos como splits o dividendos, para mantener la coherencia de los precios históricos.”	

Historia de Usuario	
Número: 23	Usuario: Equity Core de Solactive
Nombre Historia: Implementación de alertas para datos inválidos reales	
Descripción: “Si manejásemos un sistema de alertas de inválidos reales, podríamos brindarles atención directa a esos datos para el cálculo”	

Historia de Usuario	
Número: 24	Usuario: Analista de Price Data de Solactive
Nombre Historia: Reversión de validaciones manuales	
Descripción: ”Quiero poder revertir una validación manual y restaurar su estado anterior, para corregir errores en los resultados sin afectar otras validaciones y dejando registro de quién realizó la acción y cuándo.”	

Historia de Usuario	
Número: 25	Usuario: Analista de Equity Core de Solactive
Nombre Historia: Consulta de datos validados para visualización	
Descripción: “Como analista, necesito consultar la base de datos final desde la aplicación de visualización, para analizar los resultados consolidados.”	

Historia de Usuario	
Número: 26	Usuario: Analista de Equity Core de Solactive
Nombre Historia: Panel consolidado de validaciones	
Descripción: “Quiero ver en un panel el resumen de las validaciones por tipo, para entender el estado general de los datos.	

Historia de Usuario	
Número: 27	Usuario: Analista de Equity Core de Solactive
Nombre Historia: Filtros dinámicos por vendedor y tipo de activo	
Descripción: “Necesito que el sistema pueda filtrar los datos por vendedor, tipo de activo o rango de tiempo, para visualizar solo la información relevante.”	

Historia de Usuario	
Número: 28	Usuario: Analista de Equity Core de Solactive
Nombre Historia: Comparación de desempeño entre vendedores	
Descripción: “Como analista, quiero ver estadísticas de inconsistencias y frecuencia de errores por vendedor, para identificar qué proveedor presenta más fallos.”	

Historia de Usuario	
Número: 29	Usuario: Jefe de Price Data de Solactive
Nombre Historia: Visualización del historial de validaciones	
Descripción: “Como jefe de Price Data, necesito visualizar el historial de validaciones realizadas, para consultar los registros previos y mantener trazabilidad de los datos.”	

Historia de Usuario	
Número: 30	Usuario:
Nombre Historia:	
Descripción:	

Historia de Usuario	
Número: 31	Usuario: Analista de Price Data de Solactive
Nombre Historia: Consulta de datos validados históricos	
Descripción: “Como analista, necesito consultar versiones anteriores de los datos validados para comparar los cambios realizados en distintas fechas y entender el historial de cada activo.”	

Historia de Usuario	
Número: 32	Usuario: Analista de Price Data de Solactive
Nombre Historia: Panel de trazabilidad de precios	
Descripción: “Como analista, necesito visualizar la trazabilidad completa de un precio (origen, reglas aplicadas y validación final), para auditar el proceso fácilmente.”	

Historia de Usuario	
Número: 33	Usuario: Analista de Price Data de Solactive
Nombre Historia: Eliminación controlada de registros	
Descripción: “Quiero que el sistema solo permita eliminar registros de validación después de confirmar la acción, para evitar borrar información por error.”	

Historia de Usuario	
Número: 34	Usuario: Analista de Price Data de Solactive
Nombre Historia: Destacar activos con inconsistencias	
Descripción: “Sería de gran utilidad que el sistema resalte visualmente los activos con inconsistencias o validaciones incompletas, para detectarlos con rapidez.”	

Historia de Usuario	
Número: 35	Usuario: Analista de Price Data de Solactive
Nombre Historia: Exportación de gráficos y tablas desde Power B	
Descripción: “Necesito exportar las visualizaciones en PDF o Excel, para compartir reportes con otros equipos con mayor facilidad.”	

Historia de Usuario	
Número: 36	Usuario: Analista de Equity Core de Solactive
Nombre Historia: Notificación de fallos de recolección de datos	
Descripción: “Como analista, quiero recibir alertas automáticas cuando falle la recolección de datos de algún vendor, para resolverlo rápidamente.”	

Historia de Usuario	
Número: 37	Usuario: Analista de Price Data de Solactive
Nombre Historia: Registro de acciones de usuario	
Descripción: “Me gustaría que el sistema guarde automáticamente las acciones que realizamos, como validaciones o eliminaciones, junto con la fecha y el usuario, para mantener trazabilidad.”	

Historia de Usuario	
Número: 38	Usuario: Analista de Price Data de Solactive
Nombre Historia: Respaldo previo a modificaciones	
Descripción: “Quiero que el sistema genere una copia de los registros antes de realizar cualquier modificación o eliminación, para poder recuperar la información si algo sale mal.”	

Historia de Usuario	
Número: 39	Usuario: Analista de Price Data de Solactive
Nombre Historia: Gestión de incidencias en validaciones	
Descripción: “Quiero poder crear y gestionar incidencias cuando un precio quede UNVALIDATED, para registrar comentarios, adjuntos y su estado de resolución.	

Historia de Usuario	
Número: 40	Usuario: Analista de Price Data de Solactive
Nombre Historia: Edición y reversión controlada de validaciones	
Descripción: “Necesito revertir validaciones manuales y reejecutar reglas, para corregir errores manteniendo la trazabilidad, para tener un mejor orden.”	

2. Segunda Entrega

2.1. Flujo de solución

A continuación se presentan los flujos de solución planteados para tratar la problemática de validaciones de precios.

2.1.1. Opción 1

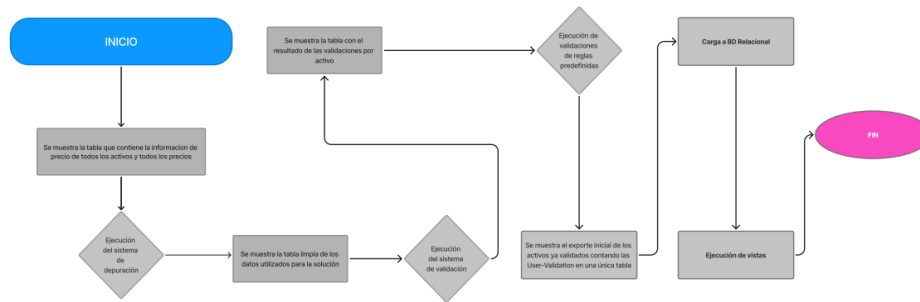


Figura 1: Opción 1 de solución: Construcción de reglas de validación bajo supervisión de la empresa

Este flujo de solución consiste en construir las reglas de validación de la mano de la empresa, para evitar validaciones incorrectas durante el proceso.

2.1.2. Opción 2

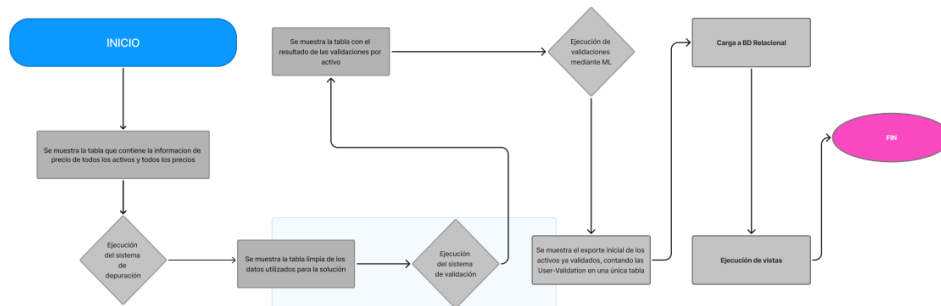


Figura 2: Opción 2 de solución: Uso de algoritmos de clasificación de Machine Learning para detectar las User-Validation y los datos no validados correctos

Este flujo de solución consiste en utilizar el algoritmo de Machine Learning para detectar la clasificación correcta sobre si es un *USER-VALIDATION* o un *UNVALIDATED*

2.1.3. Solución seleccionada

Se seleccionó la opción 1, porque esta permite evitar problemas de clasificación propios de los algoritmos de Machine Learning, lo cual puede ser costoso para la empresa y afectar su reputación directamente.

2.2. Diagrama de clases

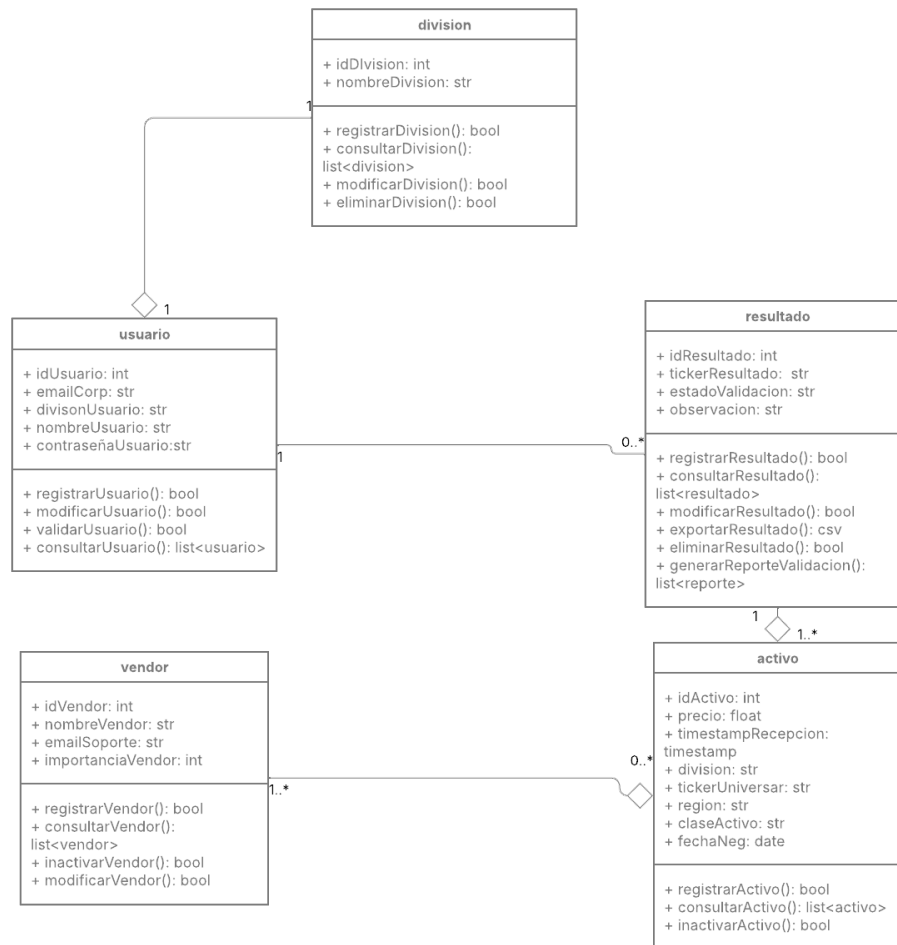


Figura 3: Diagrama de clases para la validación de precios entre activos

2.3. Modelo Entidad-Relación (MER)

Para poder visualizar al MER, acceder haciendo clic en la siguiente palabra: [MER](#)

2.4. Modelo físico

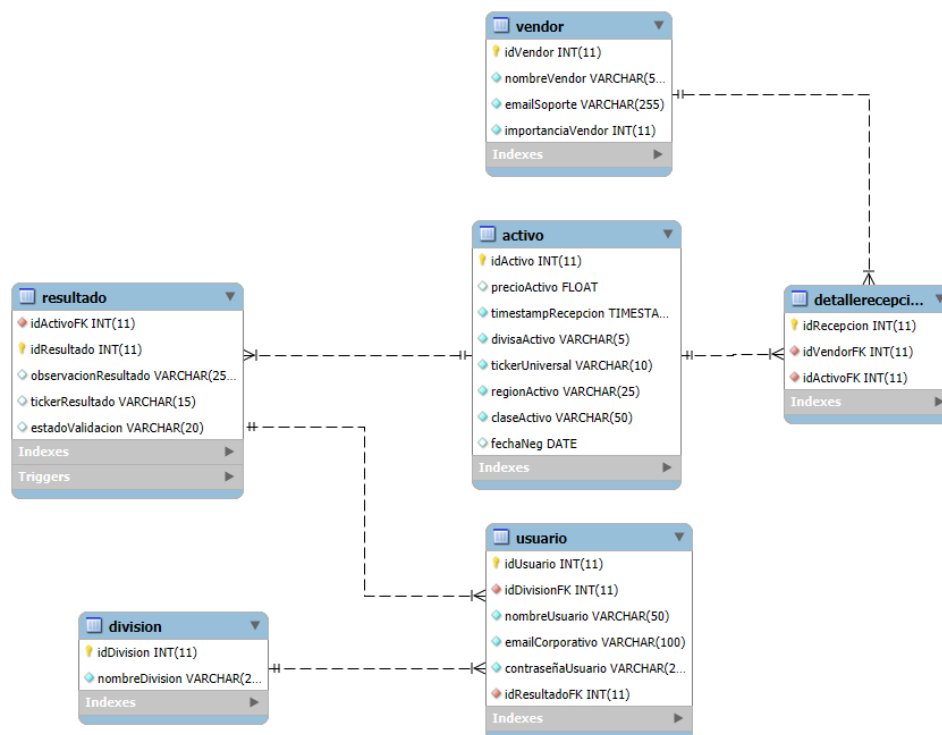


Figura 4: Modelo físico de base de datos relacional

2.5. Diccionario de datos

Nombre de la tabla	Vendor			
Descripción	Tabla que contiene la información de los diferentes vendors.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idVendor	int	—	Primary key – AUTOINCREMENT	Llave primaria del vendor.
nombre Vendor	varchar	50	NOT NULL	Nombre del vendor.
emailSoporte	varchar	255	NOT NULL	Email de contacto al área de soporte del vendor correspondiente.
importancia Vendor	int	—	NOT NULL	Nivel de importancia del vendor (1 es el más alto y 3 el más bajo).

Cuadro 1: Vendor

Nombre de la tabla	Activo			
Descripción	Tabla que contiene la información relevante de los activos registrados.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idActivo	int	–	Primary key – AUTOINCREMENT	Llave primaria del activo.
precioActivo	float	–	NOT NULL	Precio del activo.
timestamp Recepcion	timestamp	–	NOT NULL	La fecha y hora en la que fue enviada la información del activo brindada por un vendedor específico.
divisaActivo	varchar	5	NOT NULL	Moneda en la que el activo se negocia en el sistema de información de cada vendedor.
ticker Universal	varchar	10	NOT NULL	Identificador universal del activo entre los 3 vendedores de información.
regionActivo	varchar	25	NOT NULL	Región de emisión del activo.
claseActivo	varchar	50	NOT NULL	Tipo de activo negociado (p.ej., acción, bono, moneda).

Cuadro 2: Activo

Nombre de la tabla	DetalleRecepcion			
Descripción	Tabla débil que permite relacionar la información de los activos con el vendedor que la brinda.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idRecepcion	int	–	Primary key – AUTOINCREMENT	Identificador de la recepción.
idVendorFK	int	–	Foreign Key	Llave foránea del vendedor; indica qué vendedor brindó la información del activo.
idActivoFK	int	–	Foreign Key	Llave foránea del activo; permite relacionar el activo registrado con el vendedor que brindó la información.

Cuadro 3: DetalleRecepcion

Nombre de la tabla	Resultado			
Descripción	Tabla que muestra el resultado de las validaciones de la información de precios de los activos brindada por cada vendedor.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idResultado	int	–	Primary key – AUTOINCREMENT	Llave primaria del resultado; identifica un resultado para los precios brindados por los 3 vendedores para un activo en la misma fecha de negociación.
idActivoFK	int	–	Foreign Key	Llave foránea del activo; relaciona la información del activo brindada por el vendedor primario.
observacion Resultado	varchar	255	–	Observación asociada a precios invalidados o a validaciones realizadas por usuario, para especificar la razón del resultado.

Cuadro 4: Resultado

Nombre de la tabla	Usuario			
Descripción	Tabla que contiene la información relevante de los usuarios que acceden a la base de datos.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idUsuario	int	–	Primary key – AUTOINCREMENT	Llave primaria del usuario; identificador único para cada usuario.
idDivisionFK	int	–	Foreign Key – NOT NULL	Llave foránea de la división; identifica la división a la que pertenece el usuario.
nombre Usuario	varchar	50	NOT NULL	Nombre del usuario.
email Coparativo	varchar	100	NOT NULL	Email corporativo correspondiente al usuario.
contraseña Usuario	varchar	250	NOT NULL	Contraseña del usuario.
idResultado FK	int	–	Foreign Key	Llave foránea de los resultados; identifica los resultados ejecutados por el usuario.

Cuadro 5: Usuario

Nombre de la tabla	Division			
Descripción	Tabla que muestra la información relevante de las divisiones que tienen acceso a la base de datos.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idDivision	int	—	Primary key – AUTOINCREMENT	Llave primaria de la división.
nombre Division	varchar	250	NOT NULL	Nombre de las divisiones que tienen acceso a la base de datos.

Cuadro 6: Division

2.6. Ejecución de Requisitos

1. RQF_LOW11:

Ejecución:

```
1 select nombreVendor as "Nombre Vendor", importanciaVendor as "Jerarquía" from
   ↳ vendor order by importanciaVendor asc
```

Resultado:

	Nombre Vendor	Jerarquía
▶	ICE	1
	FactSet	2
	EDI	3

vendor 44 x			
Output			
Action Output			
#	Time	Action	Message
✓ 1	14:17:56	select nombreVendor as "Nombre Vendor", importanciaVendor as "Jerarquía" from vendor ord...	3 row(s) returned

2. RQF_LOW14:

Ejecucion:

```
1 call ComparacionPrecio(ticker_comparacion varchar(10), fecha_inicio
   ↳ date, in fecha_fin date, fecha_consulta date)
```

Donde los parametros del proceso son especificados por el usuario, en caso de no buscar filtrar, deberá escribir *null* en todos los parámetros.

Ejemplo de uso:

```
1 call ComparacionPrecio(null, null, null, null)
```

Resultado:

	ticker	fechaNegociacion	precioIce	precioFactset	precioEdi	idTemp	diferenciaIceFact	diferenciaIceEdi	diferenciaFactEdi	estadoPrecio
▶	DMSD	2025-08-21	2.76200008392334	2.76200008392334	2.76200008392334	1	0	0	0	Validated
	XAMS	2025-01-06	0.9318000078201294	0.9318000078201294	0.930899977684021	2	0	0.00090003	0.00090003	Semi-Validated
	XAMS	2025-12-31	97.72000122070312	97.72000122070312	97.72000122070312	3	0	0	0	Validated
	XATH	2025-01-17	10.119999885559082	10.119999885559082	10.119999885559082	4	0	0	0	Validated
	XBRN	2025-06-10	4.400000095367432	4.400000095367432	4.400000095367432	5	0	0	0	Validated
	XBRU	2025-07-28	31.299999237060547	31.299999237060547	31.299999237060547	6	0	0	0	Validated
	XCSE	2025-03-14	116	116	116	7	0	0	0	Validated
	XHEL	2025-07-02	0.6200000047683716	0.6200000047683716	0.6200000047683716	8	0	0	0	Validated
	XHEL	2025-08-22	14.34000015258789	14.34000015258789	NULL	9	0	NULL	NULL	Semi-Validated

Result 47 ×

Output

Action Output

#	Time	Action	Message
20	16:55:06	call ComparacionPrecio(null, null, null, null)	17 row(s) returned

3. RQF_LOW15:

Ejecucion:

```
1 call ValidacionManual(ticker_val varchar(5), fecha_val date, region_val
  ↳ varchar(25))
```

Donde los parametros del proceso son especificados por el usuario, en caso de no buscar filtrar, deberá escribir *null* en todos los parámetros.

Ejemplo de uso:

```
1 call ValidacionManual('XHEL','2025-08-22','EUROPE')
```

Resultado:

mensaje
▶ El resultado ya fue validado

Result 49

×

Output

Action Output

▼

#	Time	Action	Message
✓ 23	17:38:07	call ValidacionManual('XHEL','2025-08-22','EUROPE')	1 row(s) returned

4. RQF_LOW16:

Ejecución:

```
1 call ConteoValidaciones()
```

Resultado:

	Estado	count(*)
▶	Semi-Validated	2
	Validated	15

Result 91 ×			
Output			
Action Output			
#	Time	Action	Message
✓ 140	17:44:57	call ConteoValidaciones()	2 row(s) returned

5. RQF_LOW17:

Ejecución:

6. RQF_LOW20:

Ejecución:

```
1      call FiltroActivos(divisaBusqueda varchar(5), tickerBusqueda
      ↳ varchar(5), regionBusqueda varchar(5), claseActivoBusqueda
      ↳ varchar(25), fechaBusqueda date)
```

Donde los parametros del proceso son especificados por el usuario, en caso de no buscar filtrar, deberá escribir *null* en todos los parámetros.

Ejemplo de uso:

```
1      call FiltroActivos(null, 'XHEL', null, null, '2025-08-22');
```

Resultado:

	idActivo	precioActivo	timestampRecepcion	divisaActivo	tickerUniversal	regionActivo	claseActivo	fechaNeg
▶	25	14.34	2025-08-22 16:00:01	EUR	XHEL	EUROPE	SHARE	2025-08-22
	26	14.34	2025-08-22 21:45:46	EUR	XHEL	EUROPE	SHARE	2025-08-22

Result 93 ×			
Output			
Action Output			
#	Time	Action	Message
✓ 142	17:53:54	call FiltroActivos(null, 'XHEL', null, null, '2025-08-22')	2 row(s) returned

7. RQF_LOW24:

Ejecución:

```
1 delete from resultado where idResultado = int;
```

Si el número entero correspondiente al ID existe en la tabla de resultados y el estado de la validación correspondiente es igual a *User-Validation*, el trigger permitirá la eliminación, de lo contrario, la eliminación no se ejecutará y emitirá el mensaje *El resultado no es un User-Validation*.

Ejemplo de uso:

```
1 delete from resultado where idResultado = 5;
```

Resultado:

#	Time	Action	Message
144	17:57:04	delete from resultado where idResultado = 5	Error Code: 1644. El resultado no es un User Validation